ARTEFACTOS DEL MÓDULO

Punto de Control 3 (PC3)

Fase de Elaboración: arquitectura, contratos y diseño del módulo

Pacíficos Online — IF6100

INFORMACIÓN DEL SUBGRUPO

Subgrupo y módulo	SG7 — M7 — ERP contable y reportes gerenciales
Integrantes	Martin Chavez Angulo C4E332 - Coordinador Krystell Membreño Marchena C4H089 Kristy Acosta Mercado C20051 David Avila Kristel Samanta De Franco Navarro C4E644
Representante ante el Comité	Martin Chavez Agulo
Fecha de entrega	Jueves 22 de octubre de 2026 (Semana 11)
Versión del documento	[0.1 — borrador]

INSTRUCCIONES

- Este documento es la plantilla común para los 7 subgrupos. Cada subgrupo hace su copia y la nombra SG#_M#_Artefactos_PC3.
- Los textos en gris entre corchetes [así] se sustituyen por el contenido del módulo. Los recuadros azules «Cómo hacerlo» explican qué se espera; se eliminan en la versión final.
- La sección 1 se completa primero: define de qué depende el módulo, tanto de otros módulos como de los artefactos del Comité.
- Los diagramas se insertan como imagen y su archivo fuente se guarda en el repositorio.
- Todo lo que el documento menciona como código (contrato, migraciones, dobles de prueba, prototipo) debe existir en el repositorio único.

1. Dependencias del módulo

M7 — ERP contable y reportes gerenciales depende de decisiones y artefactos colectivos del Comité para definir de forma consistente sus contratos, clases, modelo de datos, interfaces y mecanismos de integración. Además, depende funcionalmente de M2, M4, M5 y M6 para obtener la información necesaria para generar registros contables y reportes.

1.1 Dependencias con los artefactos del Comité (PC3)

Cada artefacto del subgrupo usa algo que define el Comité. Si el Comité aún no lo ha publicado, el subgrupo anota el supuesto que usa y lo ajusta cuando esté disponible.

Artefacto del subgrupo	Depende de (Comité PC3)	Qué toma de ahí	Estado
2. Contrato OpenAPI	4.1 Reglas de publicación de contratos; carpeta contratos/ del repositorio; 5.5 Manejo de errores	Formato, esquema de versionado, ubicación del archivo, estructura estándar de error	Pendiente de decisión/publicación del Comité
3 y 4. Realización y clases de diseño	1.5 Vista lógica (capas); 5. Núcleo compartido	Capas internas que siguen todos los módulos; clases del núcleo para identidad, roles y aislamiento	Pendiente de cierre del Comité
5. Diagramas de interacción	4.2 y 4.3 Contratos de otros módulos; 1.6 Vista de procesos	Operaciones y eventos disponibles; qué interacciones son síncronas o por eventos	Pendiente de publicación de contratos
6. Modelo de datos y migraciones	6. Modelo de datos consolidado; 1.8 Convenciones de implementación; 1.9 Decisiones de datos	Tablas reservadas, claves compartidas, convención de nombres, uso o no de llaves foráneas entre módulos	Pendiente de consolidación
7. Patrones de diseño	3. ADR	Decisiones transversales que los patrones del módulo no deben contradecir	Pendiente de ADR
8. Prototipo de interfaz	ADR del frontend; carpeta frontend/ y componentes compartidos	React + Tailwind CSS, organización por módulo, componentes comunes	Pendiente de decisión del frontend
9. Dobles de prueba	4. Contratos publicados de otros módulos	Versión del contrato que el doble debe imitar	Pendiente de contratos definitivos

1.2 Lo que el subgrupo entrega al Comité

Insumo	Destino en el Comité	Fecha	Entregado
Lista de tablas corregida del módulo	6. Modelo de datos consolidado	Jue 1 oct	[Sí / No]
Borrador del contrato (operaciones y eventos)	4. Contratos publicados	Sáb 3 oct	[Sí / No]
Contrato OpenAPI en la carpeta contratos/	4. Contratos publicados	Pendiente de fecha definitiva del Comité	[Sí / No]

Insumo	Destino en el Comité	Fecha	Entregado
Módulo desplegado en el ambiente (implementación mínima)	7. Ambiente de integración	Durante etapa de desarrollo, antes del cierre de integración	[Sí / No]

2. Contrato formal del módulo (OpenAPI)

Descripción formal de las operaciones y eventos que el módulo ofrece, versionada y publicada en el repositorio.

Cómo hacerlo

- Escribir el contrato en formato OpenAPI (YAML) siguiendo las reglas de publicación del Comité (4.1) y usar el formato de error estándar del núcleo (5.5).
- Usar las claves compartidas con el formato del modelo canónico (id_entidad).
- Validar el archivo en un editor de OpenAPI antes de publicarlo y subirlo a la carpeta contratos/ del módulo.
- Un cambio que rompe a los consumidores genera una versión nueva; no se modifica la versión ya publicada.

2.1 Datos del contrato

Versión	v1.0.0
Archivo en el repositorio	contracts/m7-contabilidad/v1/openapi.yaml
Versión de OpenAPI	3.0.3
Estado	Borrador
Consumidores	M2, M5, M6, Núcleo y usuarios autorizados

2.2 Operaciones

Método	Ruta propuesta	Descripción	Entrada	Respuesta exitosa	Errores
GET	/api/v1/m7/pedidos/{id_pedido}/estado-contable	Consulta si un pedido ya fue contabilizado	id_pedido	200 + estado contable y asientos relacionados	401, 403, 404
GET	/api/v1/m7/pagos/{id_pago}/registro-contable	Consulta si un pago ya fue registrado contablemente	id_pago	200 + estado y id_asiento	401, 403, 404
GET	/api/v1/m7/comercios/{id_comercio}/resumen-financiero	Obtiene ingresos, costos, gastos y utilidad del comercio	id_comercio, fecha_inicio, fecha_fin	200 + resumen financiero	401, 403, 422
GET	/api/v1/m7/comercios/{id_comercio}/rentabilidad	Consulta rentabilidad por centro de costo	id_comercio, periodo	200 + ingresos, costos, utilidad y centros de costo	401, 403, 422

Método	Ruta propuesta	Descripción	Entrada	Respuesta exitosa	Errores
GET	/api/v1/m7/comercios/{id_comercio}/libro-diario	Consulta los asientos del libro diario	id_comercio, fechas, filtros	200 + lista de asientos	401, 403, 422
GET	/api/v1/m7/comercios/{id_comercio}/libro-mayor	Consulta movimientos y saldo de una cuenta	id_comercio, id_cuenta, fechas	200 + movimientos y saldo	401, 403, 404, 422
GET	/api/v1/m7/comercios/{id_comercio}/estado-resultados	Consulta el estado de resultados	id_comercio, periodo	200 + ingresos, costos, gastos y utilidad	401, 403, 422
GET	/api/v1/m7/comercios/{id_comercio}/balance-general	Consulta el balance general a una fecha	id_comercio, fecha_corte	200 + activos, pasivos y patrimonio	401, 403, 422
GET	/api/v1/m7/comercios/{id_comercio}/tablero	Obtiene indicadores del tablero ejecutivo	id_comercio, periodo	200 + indicadores financieros	401, 403, 422
GET	/api/v1/m7/comercios/{id_comercio}/cuentas	Lista el plan de cuentas	id_comercio	200 + cuentas	401, 403
POST	/api/v1/m7/comercios/{id_comercio}/cuentas	Crea una cuenta contable	datos de cuenta	201 + cuenta creada	401, 403, 409, 422
PATCH	/api/v1/m7/comercios/{id_comercio}/cuentas/{id_cuenta}	Modifica una cuenta	id_cuenta + datos modificables	200 + cuenta actualizada	401, 403, 404, 409, 422

2.3 Eventos publicados

Evento	Versión	Cuándo se publica	Carga útil (campos y tipos)
No aplica actualmente	—	M7 no tiene eventos publicados definidos en PC2	—

M7 consume eventos publicados por otros módulos, pero en PC2 no se definieron eventos propios que deba publicar. Si durante la consolidación de contratos del PC3 se identifica la necesidad de un evento como AsientoContableRegistrado, éste deberá ser aprobado por el Comité antes de incorporarse al contrato.

2.4 Esquemas expuestos

EstadoContablePedido

Campo	Tipo	Descripción
id_pedido	entero	Identificador compartido del pedido
contabilizado	booleano	Indica si la operación fue contabilizada
id_asiento_venta	entero / null	Asiento generado por la venta
id_asiento_costo	entero / null	Asiento generado por costo de ventas
fecha_contabilizacion	fecha-hora / null	Fecha del registro

RegistroContablePago

Campo	Tipo	Descripción
id_pago	entero	Identificador del pago
registrado	booleano	Indica si fue registrado
id_asiento	entero / null	Asiento relacionado
fecha_registro	fecha-hora / null	Fecha del registro

ResumenFinanciero

Campo	Tipo
id_comercio	entero
fecha_inicio	fecha
fecha_fin	fecha
ingresos	decimal
costos	decimal
gastos	decimal
utilidad	decimal

RentabilidadComercio

Campo	Tipo
id_comercio	entero
periodo	texto
ingresos	decimal
costos	decimal
utilidad	decimal
centros_costo	lista

AsientoContableResumen

Campo	Tipo
id_asiento	entero
fecha	fecha-hora
tipo	texto/enumeración
id_pedido	entero / null
id_subpedido	entero / null
id_comercio	entero
descripcion	texto
estado	texto/enumeración

DetalleAsiento

Campo	Tipo
id_cuenta_contable	entero
id_centro_costo	entero / null
debe	decimal
haber	decimal

descripcion	texto / null

2.5 Historial de versiones del contrato

Versión	Fecha	Cambio	¿Rompe consumidores?
v1.0.0	[fecha de publicación]	Publicación inicial del contrato formal de M7	No

3. Realización de casos de uso (mínimo dos)

Muestra cómo las clases del módulo cumplen un caso de uso, separando clases límite, control y entidad.

Cómo hacerlo

- Elegir al menos dos casos de uso críticos especificados en el PC2; preferir los que involucran a otros módulos.
- Clase límite: lo que interactúa con el exterior (pantalla React, controlador de la API, cliente de otro módulo). Clase control: coordina la lógica del caso de uso. Clase entidad: datos que el módulo guarda.
- Dibujar un diagrama de clases de análisis y un diagrama de comunicación o secuencia del flujo principal.
- Las clases que aparecen aquí deben reaparecer, refinadas, en el diagrama de clases de diseño (sección 4).

3.1 Realización del caso de uso [CU-002: Generar asiento automático de venta]

Caso de uso (PC2)	CU-002: Generar asiento automático de venta
Actor principal	M5- Gestión de pedidos
Módulos externos involucrados	M5- Gestión de pedidos

Estereotipo	Clase	Responsabilidad en este caso de uso
Límite	PedidoDespachadoListener	Intercepta el evento asíncrono "Pedido despachado" enviado por el módulo M5.
Límite	ContabilidadController	Sirve para consultar el libro diario, libro mayor y revisar asientos contables.
Control	ProcesarPedidoDespachadoService	Coordina la ejecución del CU-002: válida la idempotencia del evento e invoca las integraciones externas.
Control	AsientoService	Aplica la lógica contable, crea el asiento de venta y valida la partida doble.
Entidad	EventoProcesado	Almacena el historial de eventos recibidos para evitar registros duplicados.
Entidad	AsientoContable	Registra la cabecera del asiento contable de venta y conserva la trazabilidad con el pedido de M5
Entidad	DetalleAsiento	Guarda cada movimiento de débito y crédito del asiento de venta.

Estereotipo	Clase	Responsabilidad en este caso de uso
Entidad	CuentaContable	Representa la cuenta del libro mayor afectada por cada movimiento contable.

Diagrama de Clases de Análisis del CU-002: “Generar asiento automático de venta”

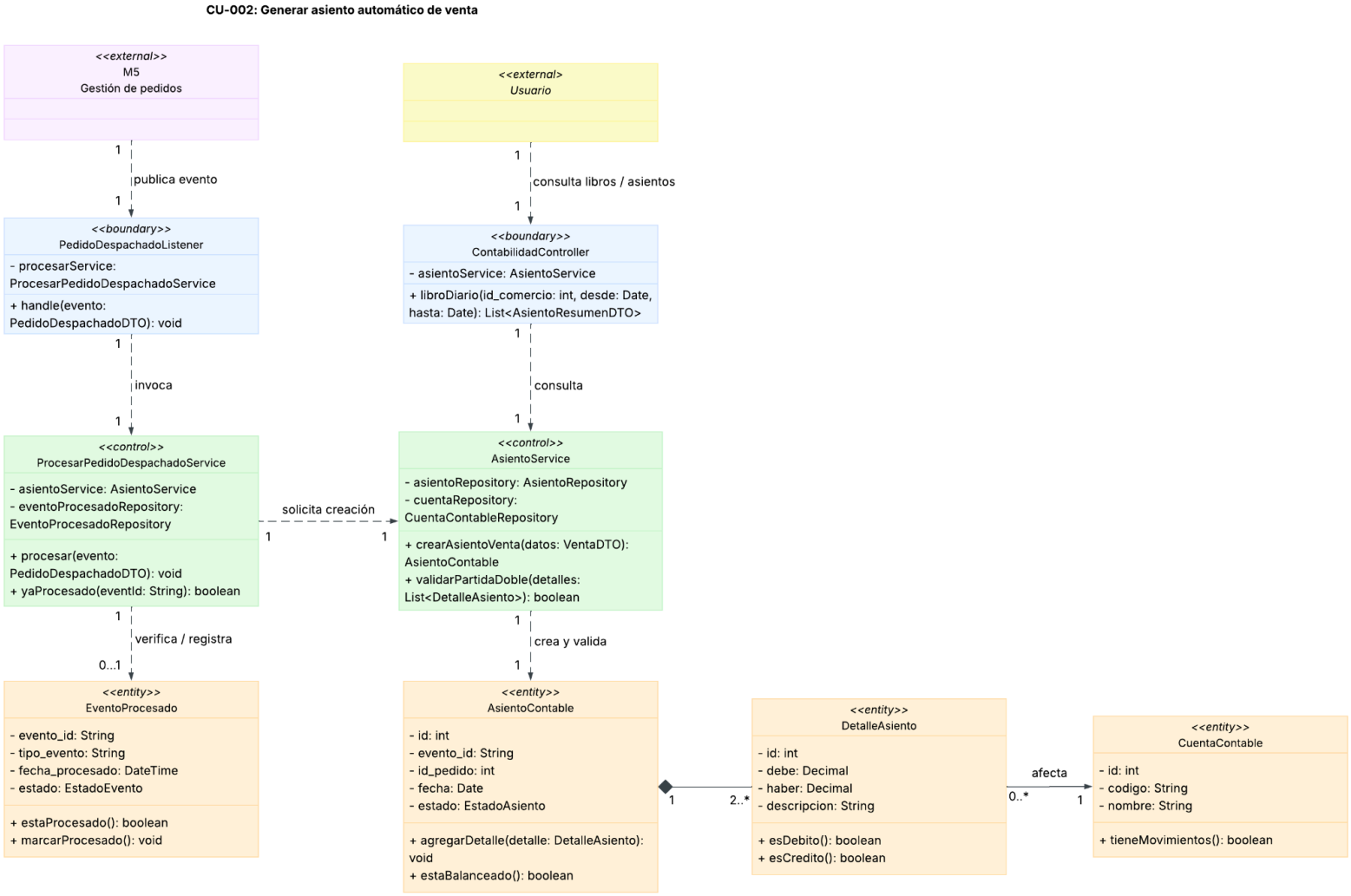
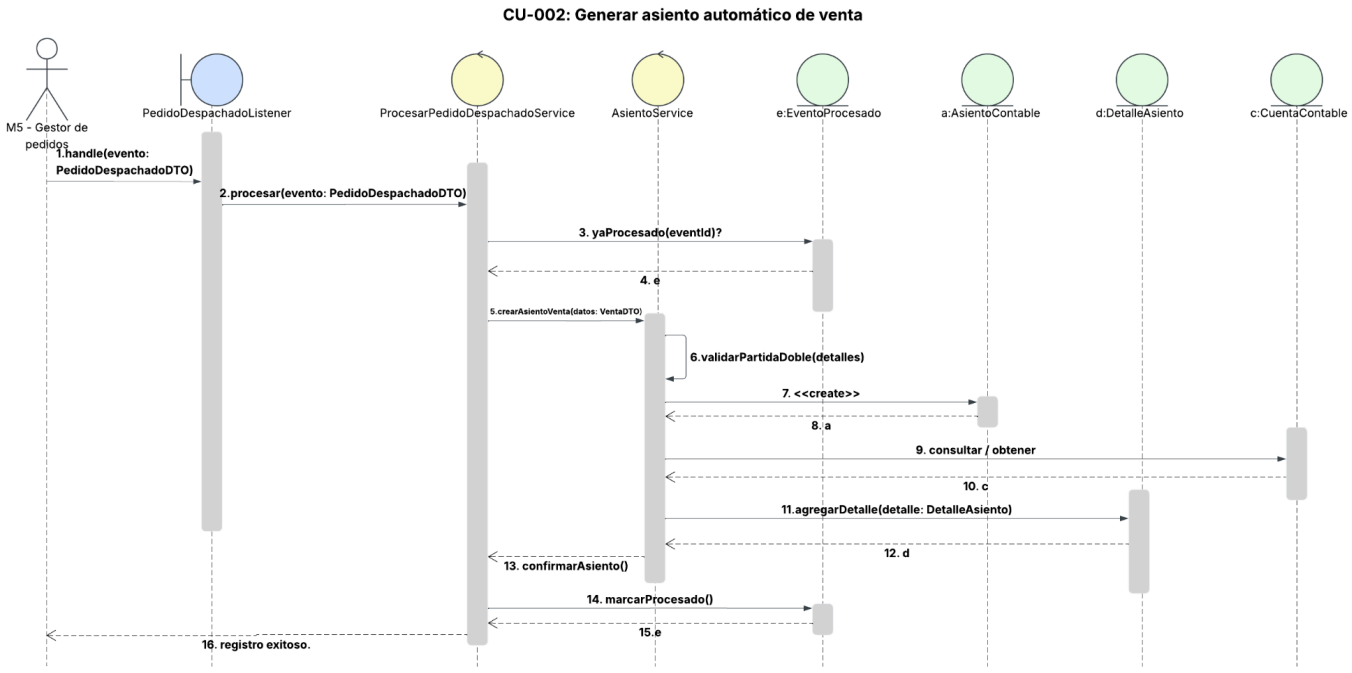


Diagrama de secuencia del CU-002: Generar asiento automático de venta



3.2 Realización del caso de uso [CU-003:Registrar asiento automático de costo de ventas]

Caso de uso (PC2)	CU-003 Registrar asiento automático de costo de ventas
Actor principal	M5- Gestión de pedidos
Módulos externos involucrados	M4- Inventario

Estereotipo	Clase	Responsabilidad en este caso de uso
Límite	ReceptorPedidoDespachado	Recibir de M5 el evento de pedido despachado que inicia el proceso
Límite	ClienteCostoInventario	Solicitar de M4 la información de costos necesaria
Control	ResgistrarCostoVentas	Coordinar el proceso de registro automático del costo de ventas
Entidad	CostoVenta	Representar el costo proporcionado por M4 para el pedido
Entidad	AsientoContable	Representar el asiento contable generado

Diagrama de Clases de Análisis del CU-003: Registrar asiento automático de costo de ventas

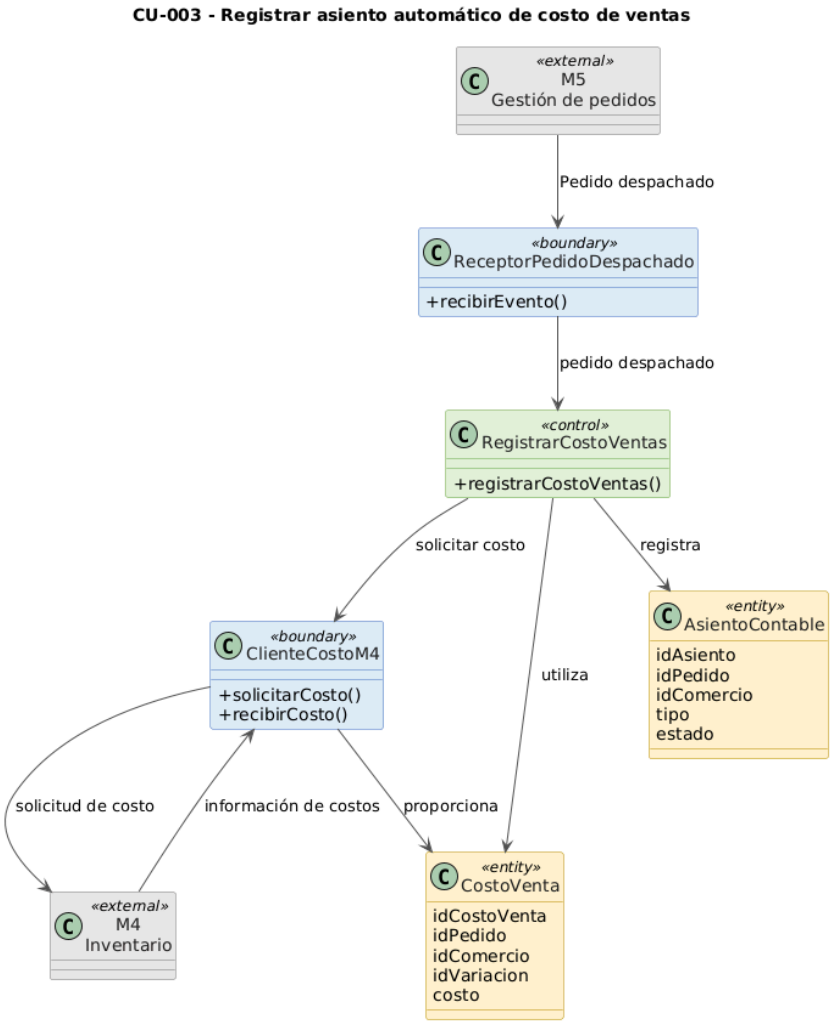
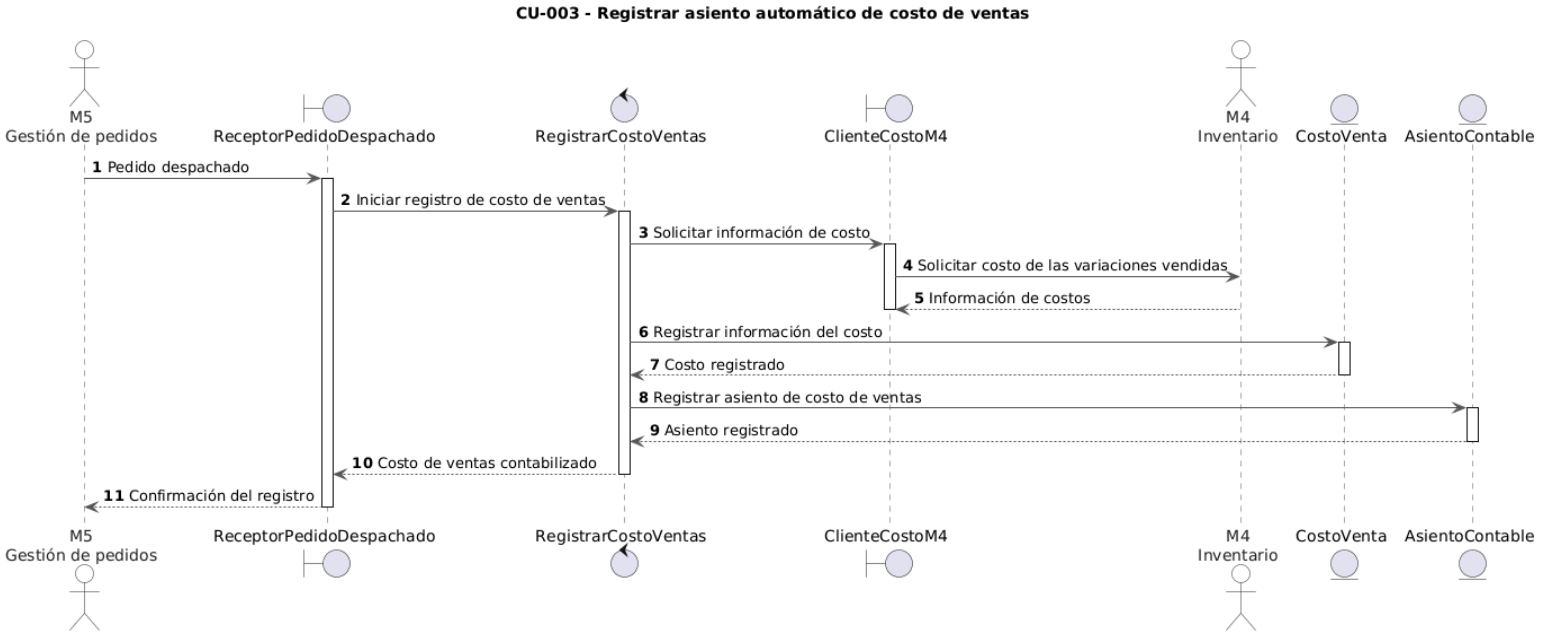


Diagrama de Secuencia del CU-003: Registrar asiento automático de costo de ventas

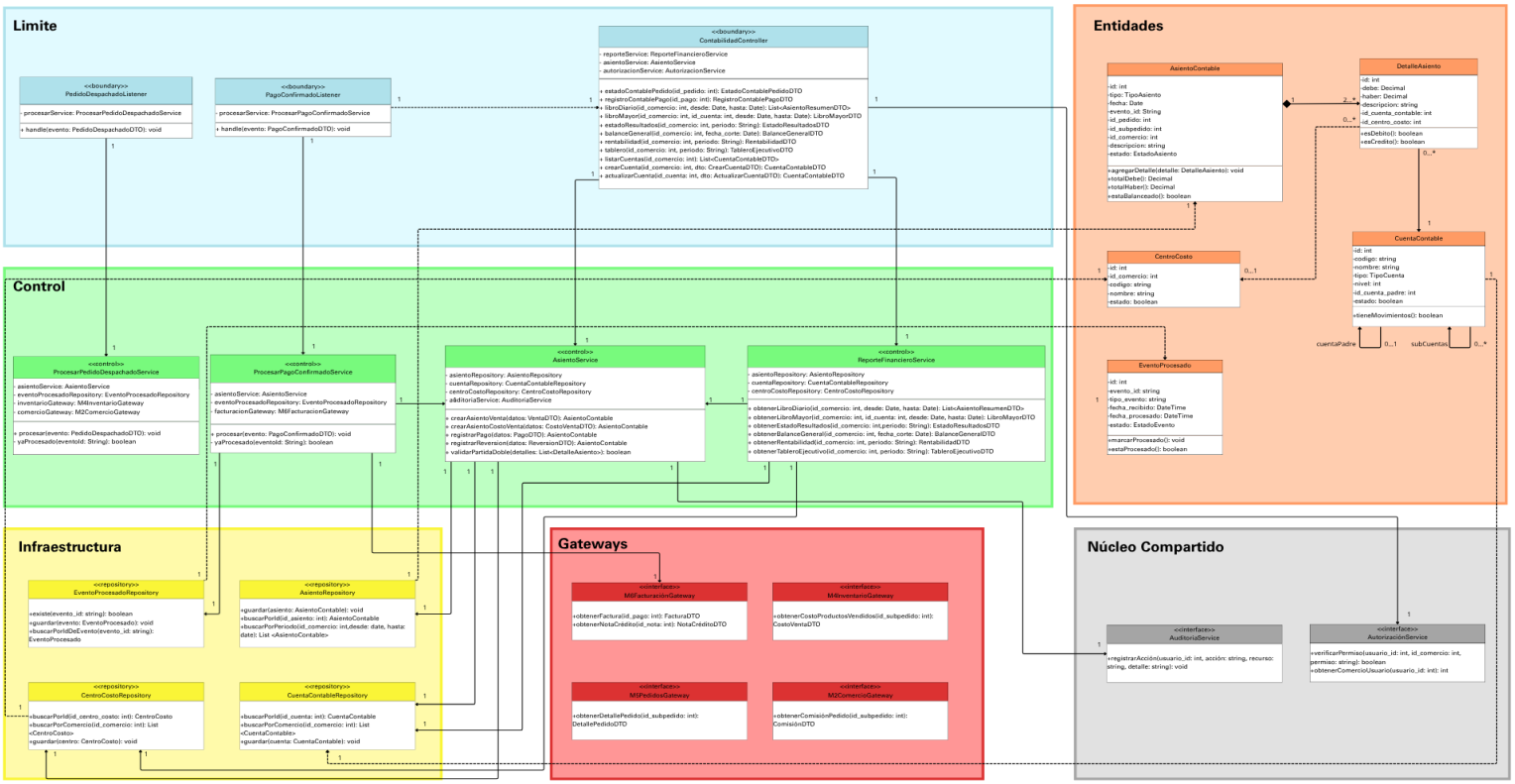


4. Diagrama de clases de diseño del módulo

Diagrama completo del módulo listo para programar: con tipos, visibilidad, firmas de métodos y navegabilidad.

Cómo hacerlo

- Refinar las clases de análisis de la sección 3 hasta el nivel de implementación en Laravel.
- Tipos: cada atributo y parámetro con su tipo (int, string, decimal, bool, fecha, enumeración, otra clase).
- Visibilidad: + pública, - privada, # protegida.
- Firmas: nombre del método, parámetros con tipo y tipo de retorno. Ejemplo de forma: + nombreMetodo(param: Tipo): TipoRetorno.
- Navegabilidad: flechas que indican qué clase conoce a cuál; incluir multiplicidades.
- Respetar las capas de la vista lógica del Comité (1.5) y usar las clases del núcleo para identidad, roles y aislamiento en lugar de reimplementarlas.



4.1 Resumen de clases

Clase	Capa	Atributos principales	Métodos principales (firma)	Relaciones y navegabilidad
PedidoDespachadoListener	Presentación	- procesarService: ProcesarPedidoDespachadoService	+ handle(evento: PedidoDespachadoDTO): void	Conoce a ProcesarPedidoDespachadoService

PagoConfirmadoListener	Presentación	- procesarService: ProcesarPagoConfirmadoService	+ handle(evento: PagoConfirmadoDTO): void	Conoce a ProcesarPagoConfirmadoService
ContabilidadController	Presentación	- reporteService: ReporteFinancieroService - asientoService: AsientoService - autorizacionService: AutorizacionService	+ estadoContablePedido(id_pedido: int): EstadoContablePedidoDTO + registroContablePago(id_pago: int): RegistroContablePagoDTO + libroDiario(id_comercio: int, desde: Date, hasta: Date): List<AsientoResumenDTO> + libroMayor(id_comercio: int, id_cuenta: int, desde: Date, hasta: Date): LibroMayorDTO + estadoResultados(id_comercio: int, periodo: String): EstadoResultadosDTO + balanceGeneral(id_comercio: int, fecha_corte: Date): BalanceGeneralDTO + rentabilidad(id_comercio: int, periodo: String): RentabilidadDTO + tablero(id_comercio: int, periodo: String): TableroEjecutivoDTO + listarCuentas(id_comerc	Conoce a ReporteFinancieroService, AsientoService y AutorizacionService

			io: int): List<CuentaContableDTO> + crearCuenta(id_comercio: int, dto: CrearCuentaDTO): CuentaContableDTO + actualizarCuenta(id_cuenta: int, dto: ActualizarCuentaDTO): CuentaContableDTO	
ProcesarPedidoDespachadoService	Aplicación	- asientoService: AsientoService - eventoProcesadoRepository: EventoProcesadoRepository - inventarioGateway: M4InventarioGateway - comercioGateway: M2ComercioGateway	+ procesar(evento: PedidoDespachadoDTO): void - yaProcesado(eventoId: String): boolean	Conoce a AsientoService, EventoProcesadoRepository, M4InventarioGateway y M2ComercioGateway
ProcesarPagoConfirmadoService	Aplicación	- asientoService: AsientoService - eventoProcesadoRepository: EventoProcesadoRepository - facturacionGateway: M6FacturacionGateway	+ procesar(evento: PagoConfirmadoDTO): void - yaProcesado(eventoId: String): boolean	Conoce a AsientoService, EventoProcesadoRepository y M6FacturacionGateway

AsientoService	Aplicación	- asientoRepository: AsientoRepository - cuentaRepository: CuentaContableRepository - centroCostoRepository: CentroCostoRepository - auditoriaService: AuditoriaService	+ crearAsientoVenta(datos: VentaDTO): AsientoContable + crearAsientoCostoVenta(datos: CostoVentaDTO): AsientoContable + registrarPago(datos: PagoDTO): AsientoContable + registrarReversion(datos: ReversionDTO): AsientoContable + validarPartidaDoble(detalles: List<DetalleAsiento>): boolean	Conoce a los repositorios de asiento, cuenta y centro de costo, además de AuditoriaService
ReporteFinancieroService	Aplicación	- asientoRepository: AsientoRepository - cuentaRepository: CuentaContableRepository - centroCostoRepository: CentroCostoRepository	+ obtenerLibroDiario(id_comercio: int, desde: Date, hasta: Date): List<AsientoResumenDTO> + obtenerLibroMayor(id_comercio: int, id_cuenta: int, desde: Date, hasta: Date): LibroMayorDTO + obtenerEstadoResultados(id_comercio: int, periodo: String): EstadoResultadosDTO + obtenerBalanceGeneral(id_comercio: int, fecha_corte: Date): BalanceGeneralDTO + obtenerRentabilidad(id_	Conoce a AsientoRepository, CuentaContableRepository y CentroCostoRepository

			comercio: int, periodo: String): RentabilidadDTO + obtenerTableroEjecutivo (id_comercio: int, periodo: String): TableroEjecutivoDTO	
AsientoContable	Dominio	- id: int - tipo: TipoAsiento - fecha: Date - evento_id: String - id_pedido: int - id_subpedido: int - id_comercio: int - descripcion: String - estado: EstadoAsiento	+ agregarDetalle(detalle: DetalleAsiento): void + totalDebe(): Decimal + totalHaber(): Decimal + estaBalanceado(): boolean	Composición con DetalleAsiento: 1 → 2..*
DetalleAsiento	Dominio	- id: int - debe: Decimal - haber: Decimal - descripcion: String - id_cuenta_contable: int - id_centro_costo: int	+ esDebito(): boolean + esCredito(): boolean	Pertenece a AsientoContable; conoce a CuentaContable (1) y opcionalmente a CentroCosto (0..1)
CuentaContable	Dominio	- id: int - codigo: String - nombre: String - tipo: TipoCuenta - nivel: int - id_cuenta_padre: int - estado: boolean	+ tieneMovimientos(): boolean	Puede tener 0..1 cuenta padre y 0..* subcuentas; recibe 0..* detalles contables
CentroCosto	Dominio	- id: int - id_comercio: int - codigo: String - nombre: String - estado: boolean	—	Puede estar asociado con 0..* DetalleAsiento; cada detalle usa 0..1 centro de costo
EventoProcesado	Dominio	- id: int - evento_id: String - tipo_evento: String - fecha_recibido: DateTime - fecha_procesado:	+ marcarProcesado(): void + estaProcesado(): boolean	Persistido mediante EventoProcesadoRepository; utilizado para idempotencia

		DateTime - estado: EstadoEvento		
AsientoRepository	Infraestructura	—	+ guardar(asiento: AsientoContable): void + buscarPorId(id_asiento: int): AsientoContable + buscarPorPeriodo(id_comercio: int, desde: Date, hasta: Date): List<AsientoContable>	Administra/persiste AsientoContable; usado por servicios de aplicación
CuentaContableRepository	Infraestructura	—	+ buscarPorId(id_cuenta: int): CuentaContable + buscarPorComercio(id_comercio: int): List<CuentaContable> + guardar(cuenta: CuentaContable): void	Administra CuentaContable; usado por AsientoService y ReporteFinancieroService
CentroCostoRepository	Infraestructura	—	+ buscarPorId(id_centro_costo: int): CentroCosto + buscarPorComercio(id_comercio: int): List<CentroCosto> + guardar(centro: CentroCosto): void	Administra CentroCosto; usado por servicios de aplicación
EventoProcesadoRepository	Infraestructura	—	+ existe(evento_id: String): boolean + guardar(evento: EventoProcesado): void + buscarPorIdDeEvento(evento_id: String): EventoProcesado	Administra EventoProcesado; usado por los procesadores de eventos

4.2 Clases del núcleo y de otros módulos que se usan

Clase o interfaz externa	Origen	Para qué la usa el módulo
AuditoriaService	Núcleo	Registrar acciones en el sistema.
AutorizaciónService	Núcleo	Verificar permisos de usuario y obtener el comercio.
FacturaciónGateway	M6	Obtener las facturas y la nota de crédito.
InventarioGateway	M4	Obtener el costo de los productos que fueron vendidos.
PedidosGateway	M5	Obtener el detalle de los pedidos.
ComercioGateway	M2	Obtener comisión de los pedidos.

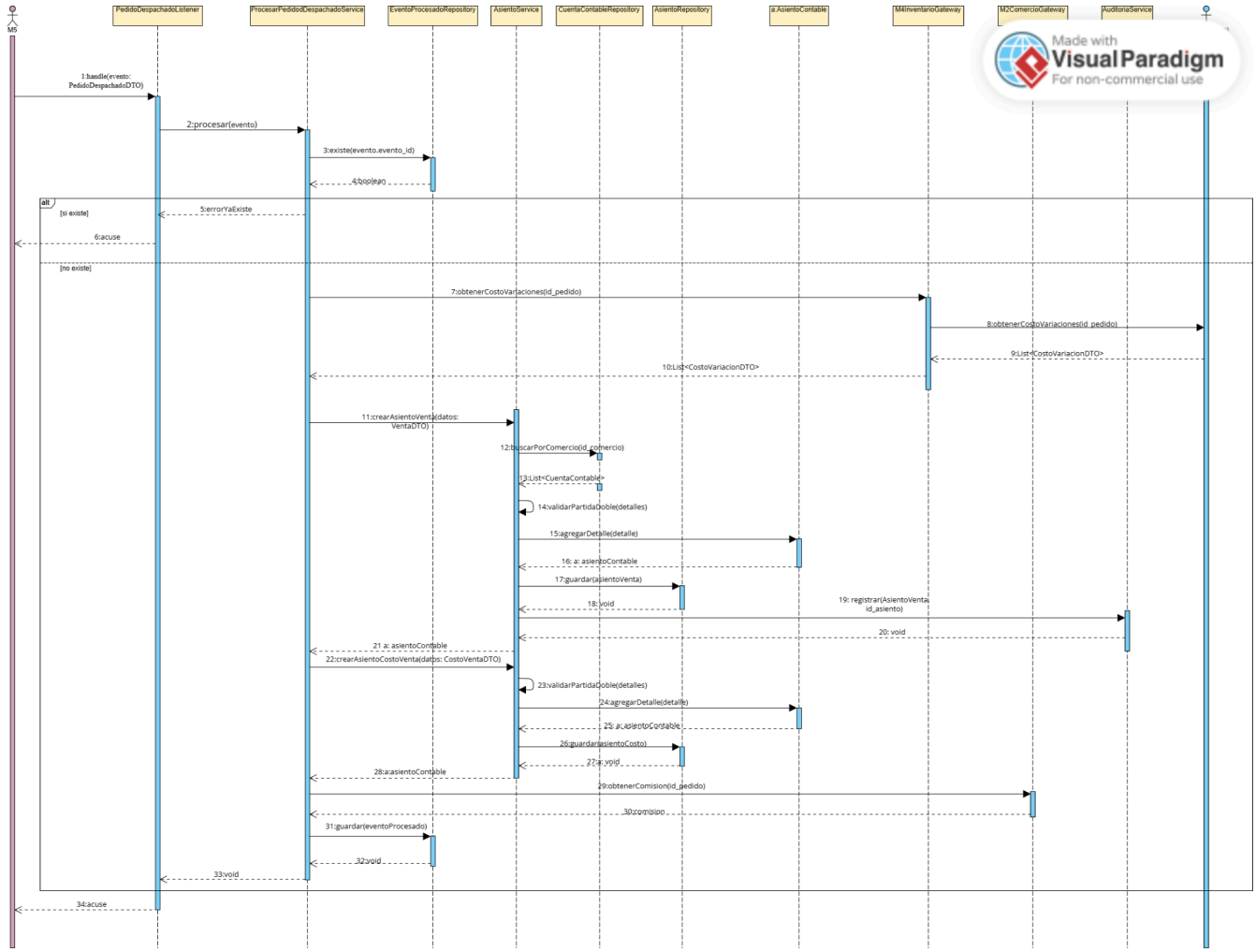
5. Diagramas de interacción actualizados

Diagramas de secuencia de los casos de uso del PC2 actualizados con las clases de diseño, incluyendo las llamadas a otros módulos.

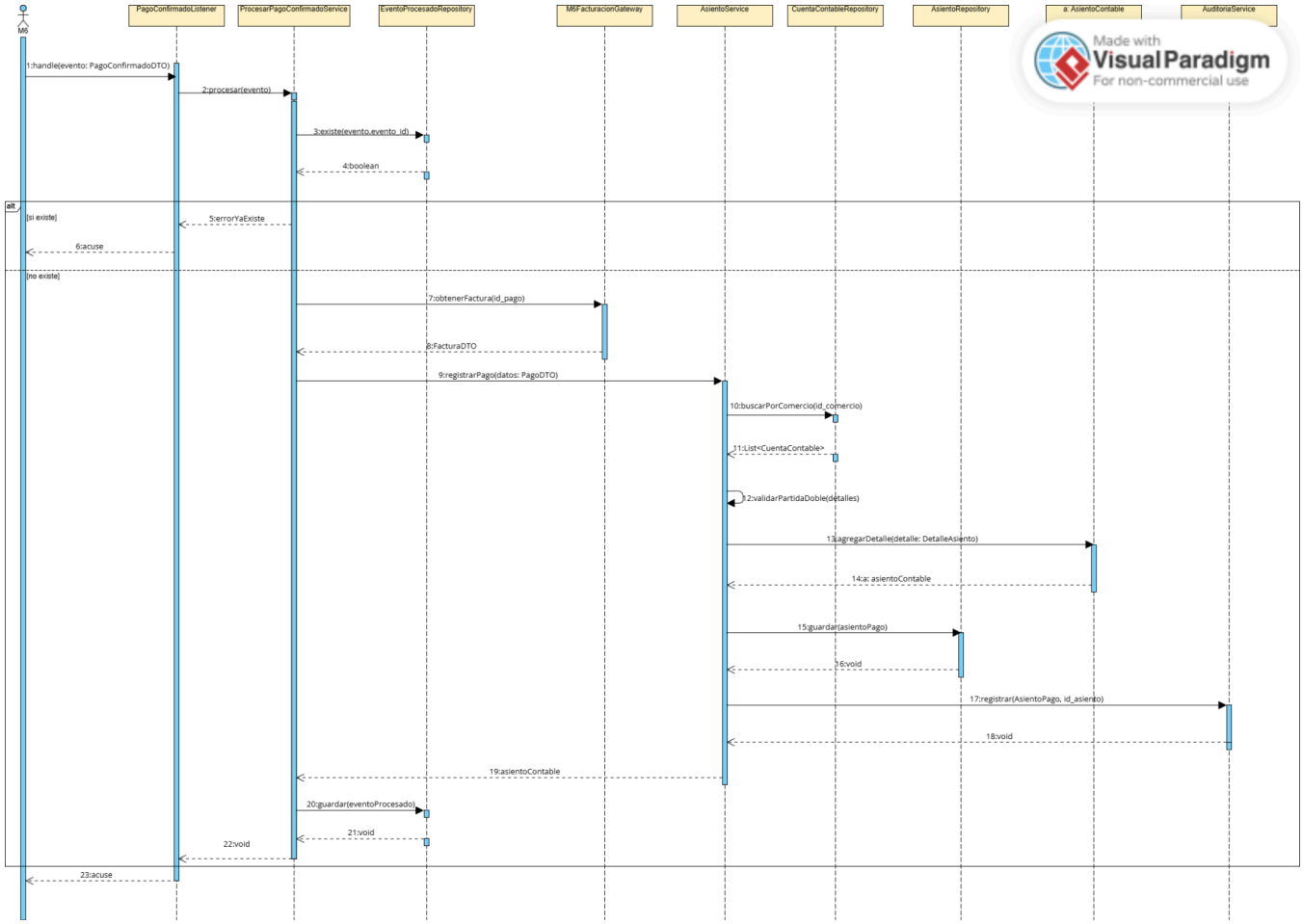
Cómo hacerlo

- Tomar los diagramas de interacción del PC2 y reemplazar los objetos de análisis por las clases de diseño de la sección 4.
- Cada mensaje debe corresponder a un método que exista en el diagrama de clases, con su firma.
- Las llamadas a otro módulo se muestran a través de su contrato (operación o evento), nunca accediendo a sus tablas o clases internas.
- Mostrar también las llamadas al núcleo (autenticación, verificación de permisos, bitácora) y los flujos de error relevantes.

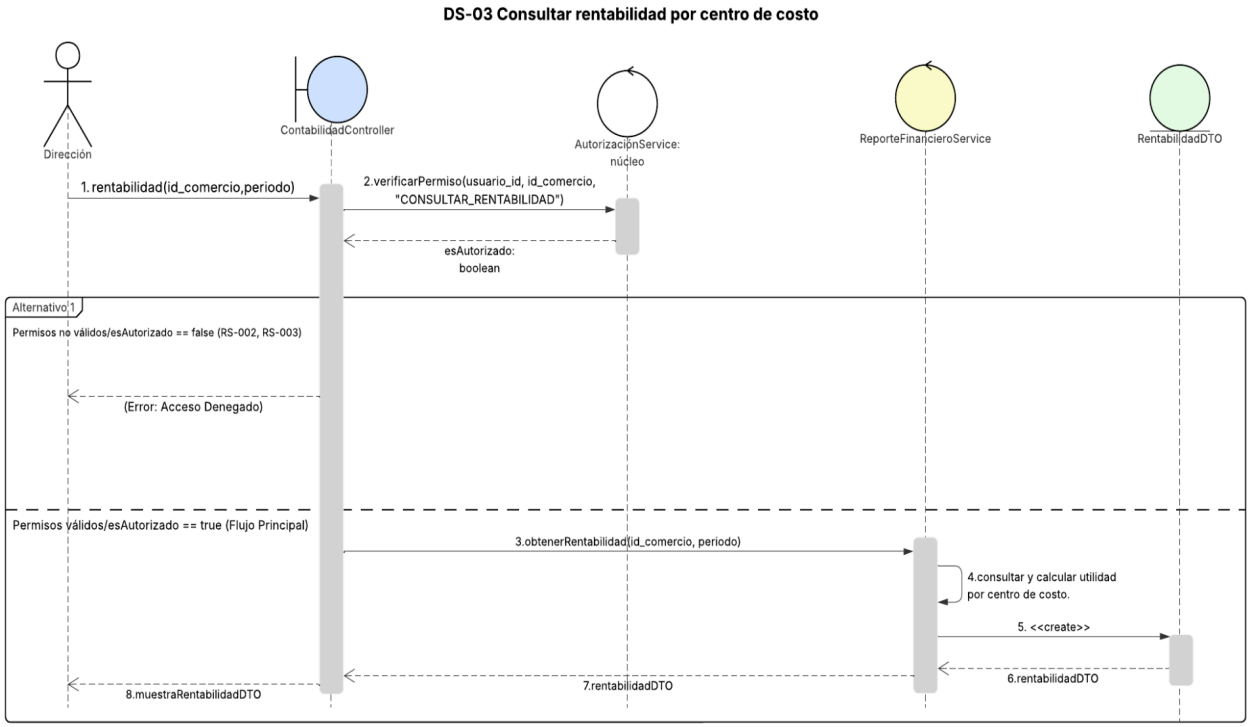
Diagrama	Caso de uso	Módulos externos llamados	Operación o evento del contrato usado
DS-01	CU-01 y CU-02 *(Generar asiento de venta y costo)*	M5 (Pedidos), M4 (Inventario), M2 (Comercios) y Núcleo	Evento (M5): `PedidoDespachado` (v1.0) Operación (M4): `obtenerCostoVariaciones(id_pedido)` (v1.0) Operación (M2): `obtenerComision(id_pedido)` (v1.0) Núcleo: `registrar(Asiento, asiento_id)`
DS-02	CU *Registrar asiento de pago*	M6 (Pagos y Facturación) y Núcleo	Evento (M6): `PagoConfirmado` (v1.0) Operación (M6): `obtenerFactura(id_pago)` Núcleo: `registrar(AsientoPago, id_asientos)`
DS-03	CU-03	Núcleo compartido	`verificarPermiso(usuario_id: int, id_comercio: int, permiso: string): boolean`


MS
PedidoDespachadoListener
ProcesarPedidodDespachadoService
EventoProcesadoRepository
AsientoService
CuentaContableRepository
AsientoRepository
a:AsientoContable
M4InventarioGateway
M2ComercioGateway
AuditoriaService
1:handle(evesto: PedidoDespachadoDTO)
2:procesar(evento)
3:existe(evento.evento_id)
4:boolean
alt
[si existe]
5:errorYaExiste
6:acuse
[no existe]
7:obtenerCostoVariaciones(id_pedido)
8:obtenerCostoVariaciones(id_pedido)
9:List<CostoVariacionDTO>
10:List<CostoVariacionDTO>
11:crearAsientoVenta(datos: VentaDTO)
12:buscarPorComercio(id_comercio)
13:List<CuentaContable>
14:validarPartidaDoble(detalles)
15:agregarDetalle(detalle)
16: a: asientoContable
17:guardar(asientoVenta)
18:void
19: registrar(AsientoVenta id_asiento)
20: void
21 a: asientoContable
22:crearAsientoCostoVenta(datos: CostoVentaDTO)
23:validarPartidaDoble(detalles)
24:agregarDetalle(detalle)
25: a: asientoContable
26:guardar(asientoCosto)
27:a: void
28:a:asientoContable
29:obtenerComision(id_pedido)
30:comision
31:guardar(eventoProcesado)
32:void
33:void
34:acuse

DS-01


M6
PagoConfirmadoListener
ProcesarPagoConfirmadoService
EventoProcesadoRepository
M6FacturacionGateway
AsientoService
CuentaContableRepository
AsientoRepository
a: AsientoContable
AuditoriaService
Made with VisualParadigm For non-commercial use
1:handle(evento: PagoConfirmadoDTO)
2:procesar(evento)
3:existe(evento.evento_id)
4:boolean
alt
[si existe]
5:errorYaExiste
6:acuse
[no existe]
7:obtenerFactura(id_pago)
8:FacturaDTO
9:registrarPago(datos: PagoDTO)
10:buscarPorComercio(id_comercio)
11:List<CuentaContable>
12:validarPartidaDoble(detalles)
13:agregarDetalle(detalle: DetalleAsiento)
14:a: asientoContable
15:guardar(asientoPago)
16:void
17:registrar(AsientoPago, id_asiento)
18:void
19:asientoContable
20:guardar(eventoProcesado)
21:void
22:void
23:acuse

DS-02



DS-03

6. Modelo de datos del módulo

Modelo en tercera forma normal, implementado como migraciones de Laravel, con restricciones de unicidad y verificación que implementan las reglas de negocio. Se entrega además el SQL resultante.

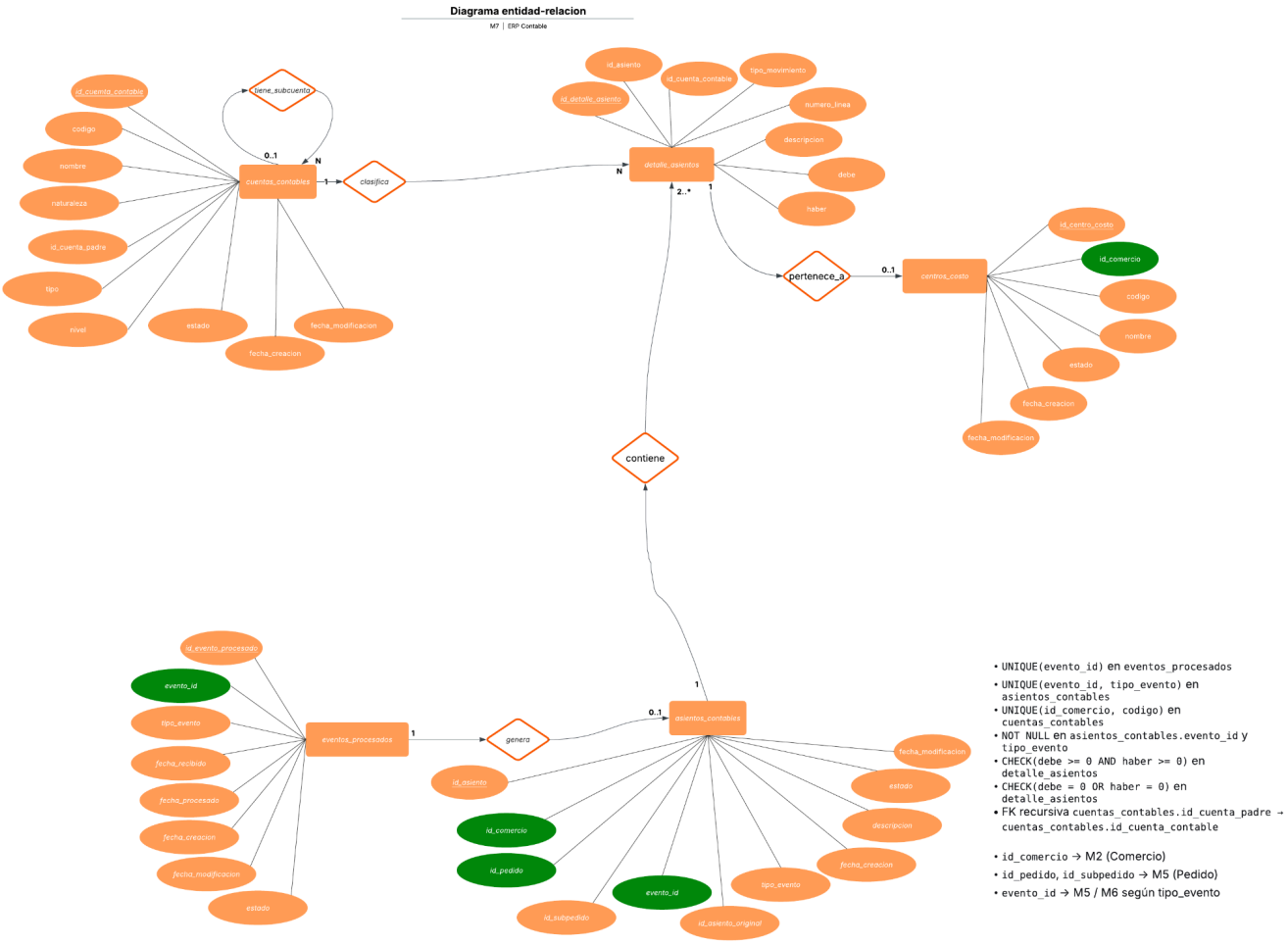
Cómo hacerlo

- Usar solo las tablas reservadas por el módulo en el PC2, con los nombres corregidos (minúscula, sin tildes, guion bajo) y las claves con formato id_entidad.
- Las referencias a entidades de otros módulos se guardan solo como clave compartida, según la decisión del Comité (1.9) sobre llaves foráneas entre módulos.
- Revisar cada tabla contra 1FN, 2FN y 3FN y justificar el resultado.
- Cada regla de negocio que se pueda expresar en la base de datos se implementa como restricción (unique, check, not null, foreign key interna) dentro de la migración.
- Para obtener el SQL resultante se puede usar php artisan migrate --pretend, o exportar el esquema de la base de datos una vez aplicadas las migraciones.

6.1 Tablas del módulo

Tabla	Clave primaria	Claves compartidas que guarda	Descripción

cuentas_contables	id_cuenta_contable	id_comercio	Almacena el plan jerárquico de cuentas utilizado por cada comercio. Incluye nivel, id_cuenta_padre para la jerarquía y naturaleza (deudora/acreedora).
asientos_contables	id_asiento	id_comercio, id_pedido, id_subpedido	Almacena el encabezado y la trazabilidad de cada asiento generado por los eventos contables. Incluye id_asiento_original para los asientos de reversión.
detalle_asientos	id_detalle_asiento	evento_id (M5/M6)	Almacena las líneas débito y crédito de cada asiento; cada línea registra debe, haber y tipo_movimiento (débito/crédito), y se relaciona con cuentas y centros de costo.
centros_costo	id_centro_costo	id_comercio	Permite clasificar movimientos contables y obtener rentabilidad por centro de costo.
eventos_procesados	id_evento_procesado	evento_id(M5/M6)	Registra los eventos recibidos para evitar que una misma notificación produzca asientos duplicados.



6.2 Verificación de tercera forma normal

Tabla	1FN (valores atómicos)	2FN (sin dependencias parciales)	3FN (sin dependencias transitivas)	Observaciones
cuentas_contables	Cumple	Cumple	Cumple	Cada campo almacena un único valor. La PK es simple y los atributos codigo, nombre, naturaleza, tipo, nivel, id_cuenta_padre, estado, fecha_creacion y fecha_modificacion dependen directamente de ella. La jerarquía se representa mediante

				id_cuenta_padre, que es una FK recursiva.
asientos_contables	Cumple	Cumple	Cumple	Cada registro representa un único asiento. Todos sus atributos describen directamente al asiento identificado por id_asiento; los datos externos se mantienen únicamente como identificadores compartidos.
detalle_asientos	Cumple	Cumple	Cumple	Cada fila representa una única línea de un asiento. El importe, descripción, cuenta y centro de costo dependen de id_detalle_asiento. No se repiten los datos descriptivos de las cuentas. tipo_movimiento es un atributo derivable de debe/haber que se conserva por eficiencia de consulta.
centros_costo	Cumple	Cumple	Cumple	Cada centro posee valores atómicos y sus atributos dependen directamente de id_centro_costo.
eventos_procesados	Cumple	Cumple	Cumple	Cada registro representa un evento recibido. Tipo, fechas y estado dependen directamente de id_evento_procesado, mientras que evento_id identifica de forma única el evento externo.

6.3 Restricciones que implementan reglas de negocio

Regla de negocio	Tabla	Tipo de restricción	Definición	Nombre en la migración
RN-008: M7 no duplica asientos de eventos ya procesados	eventos_procesados	UNIQUE	evento_id no puede repetirse	uq_eventos_procesados_evento
RN-009: Los registros contables mantienen trazabilidad con su operación	asientos_contables	NOT NULL	Todo asiento conserva evento_id y tipo_evento	nn_asiento_evento_tipo
RN-009	asientos_contables	UNIQUE	La combinación evento_id + tipo_evento no puede generar dos asientos equivalentes	uq_asiento_evento_tipo
Regla estructural de partida contable	detalle_asientos	CHECK	debe >= 0 AND haber >= 0	chk_detalle_importes_no_negativos
Regla estructural de partida contable	detalle_asientos	CHECK	Una línea debe registrar débito o crédito, pero no ambos	chk_detalle_debe_o_haber
Integridad interna	detalle_asientos	FK	id_asiento debe existir en asientos_contables	fk_detalle_asiento

Integridad interna	detalle_asientos	FK	id_cuenta_contable debe existir en cuentas_contables	fk_detalle_cuenta
Integridad interna	detalle_asientos	FK nullable	Si existe id_centro_costo, debe corresponder a un centro válido	fk_detalle_centro_costo
Plan de cuentas por comercio	cuentas_contables	UNIQUE	Un comercio no puede tener dos cuentas con el mismo código	uq_cuenta_comercio_codigo
RN-010: información financiera aislada por comercio	Varias	Política de aplicación / Núcleo	Las consultas usan el id_comercio autorizado por el Núcleo	politica_aislamiento_comercio
RN-012: acciones sobre datos contables quedan auditadas	Varias	Integración con Núcleo	Creaciones y modificaciones sensibles invocan AuditoriaService	auditoria_datos_contables
Jerarquía de cuentas	cuentas_contables	FK recursiva	id_cuenta_padre debe existir en cuentas_contables (nullable)	fk_cuenta_padre
Reversión de asiento	asientos_contables	FK recursiva	id_asiento_original debe existir en asientos_contables (nullable)	fk_asiento_original
Tipo de cuenta válido	cuentas_contables	CHECK	tipo IN (‘activo’, ‘pasivo’, ‘patrimonio’, ‘ingreso’, ‘gasto’, ‘costo’)	chk_cuenta_tipo

Tipo de movimiento valido	detalle_asientos	CHECK	tipo_movimiento IN (‘debito’, ‘credito’)	chk_detalle_tipo_movimiento
Nivel positivo	cuentas_contables	CHECK	nivel >= 1	chk_cuenta_nivel
Estado de asiento valido	asientos_contables	CHECK	estado IN (‘registrado’, ‘reservado’)	chk_asiento_estado
Código único por comercio	centros_costo	UNIQUE	(id_comercio, codigo) único	uq_centro_comercio_codigo
Coherencia tipo_movimiento	detalle_asientos	CHECK	(tipo_movimiento = ‘debito’ AND debe > 0 AND haber = 0) OR (tipo_movimiento = ‘credito’ AND haber > 0 AND debe = 0)	chk_detalle_tipo_coherente
Evento externo obligatorio	eventos_procesados	NOT NULL	evento_id y tipo_evento no nulos	nn_evento_procesado_externo
Naturaleza valida	cuentas_contables	CHECK	naturaleza IN (‘deudora’, ‘acreedora’)	chk_cuenta_naturaleza

6.4 Migraciones

Archivo de migración	Tabla	Descripción
create_cuentas_contables_table.php	cuentas_contables	Crea el catálogo jerárquico de cuentas por comercio.
create_centros_costo_table.php	centros_costo	Crea los centros de costo utilizados para segmentar movimientos financieros.
create_asientos_contables_table.php	asientos_contables	Crea los encabezados de los asientos y almacena su trazabilidad con los eventos externos.
create_detalle_asientos_table.php	detalle_asientos	Crea las líneas débito/crédito asociadas a cada asiento, cuenta y centro de costo.
create_eventos_procesados_table.php	eventos_procesados	Registra eventos ya recibidos para implementar idempotencia y evitar procesamiento duplicado.

6.5 SQL resultante

Pendiente a que el comité lo determine.
[Pegar aquí el SQL generado o indicar la ruta del archivo .sql en el repositorio.]

7. Patrones de diseño aplicados (mínimo tres)

Cómo hacerlo

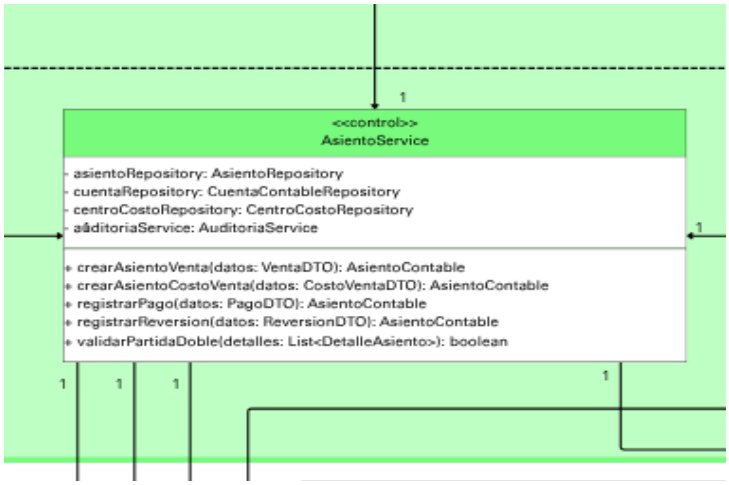
- Aplicar patrones que resuelvan un problema real del módulo, no agregarlos por cumplir: la justificación debe nombrar ese problema.
- Indicar en qué clases del diagrama de diseño (sección 4) se ve cada patrón.
- Verificar que no contradigan los ADR del Comité. Por ejemplo, un patrón para aislar la pasarela de pago o el acceso a otro módulo debe respetar la comunicación por contrato.
- Referencias habituales en Laravel: Repository, Strategy, Adapter, Observer, Factory, Facade, State; se pueden usar otros.

7.1 Strategy

Categoría	Comportamiento
Problema del módulo que resuelve	El módulo M7 debe aplicar distintas reglas de contabilización según la transacción procesada. La lógica para registrar una venta, un costo de venta, un pago o una reversión es diferente. Implementar toda esta lógica en una única

	clase dificultaría el mantenimiento y la incorporación de nuevas reglas contables.
Dónde se aplica	AsientoService AsientoContable
Justificación	El patrón Strategy permite encapsular las distintas reglas de contabilización en estrategias independientes. Esto facilita agregar nuevos tipos de transacciones sin modificar la lógica existente y mantiene separadas las reglas de negocio para ventas, costos de venta, pagos y reversiones.
Alternativa descartada	Implementar toda la lógica de contabilización mediante múltiples estructuras if-else dentro de AsientoService. Esta alternativa fue descartada porque aumenta el acoplamiento, dificulta el mantenimiento y complica la incorporación de nuevas reglas contables.

[Insertar fragmento del diagrama de clases donde se ve el patrón.]



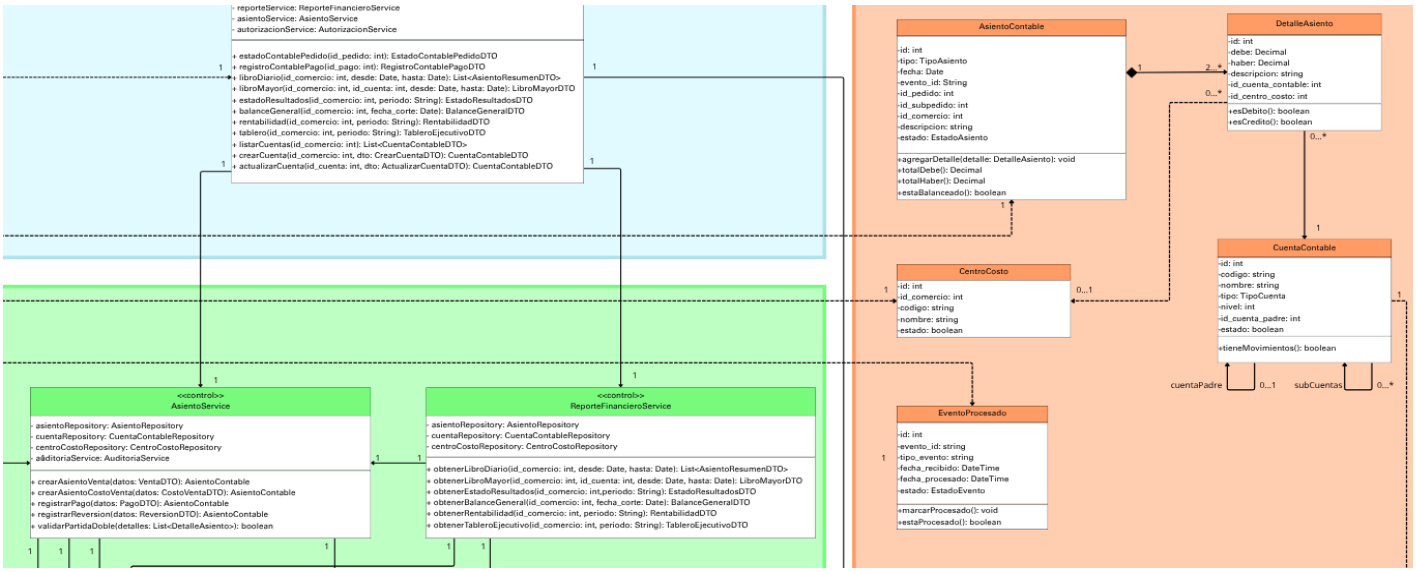
Las operaciones crearAsientoVenta(), crearAsientoCostoVenta(), registrarPago() y registrarReversion() representan distintas estrategias de contabilización según el tipo de transacción procesada.

7.2 Factory Method

Categoría	Creacional
Problema del módulo que resuelve	El módulo M7 debe crear distintos asientos contables a partir de las transacciones recibidas desde otros módulos del sistema. Cada tipo de transacción requiere una configuración específica de cuentas y detalles contables. Centralizar la creación de estos objetos evita duplicación de lógica y facilita la extensión del sistema.
Dónde se aplica	AsientoService AsientoContable DetalleAsiento
Justificación	El patrón Factory Method se propone en AsientoService para centralizar la creación de objetos AsientoContable y sus detalles asociados según el tipo de transacción procesada. Esto permite encapsular la lógica de creación y facilita la incorporación de nuevos tipos de asientos

	contables sin afectar otros componentes del módulo.
Alternativa descartada	Crear directamente los objetos AsientoContable y DetalleAsiento desde múltiples servicios del sistema. Esta alternativa fue descartada porque distribuye la lógica de creación de objetos y dificulta el mantenimiento y la consistencia de las reglas contables.

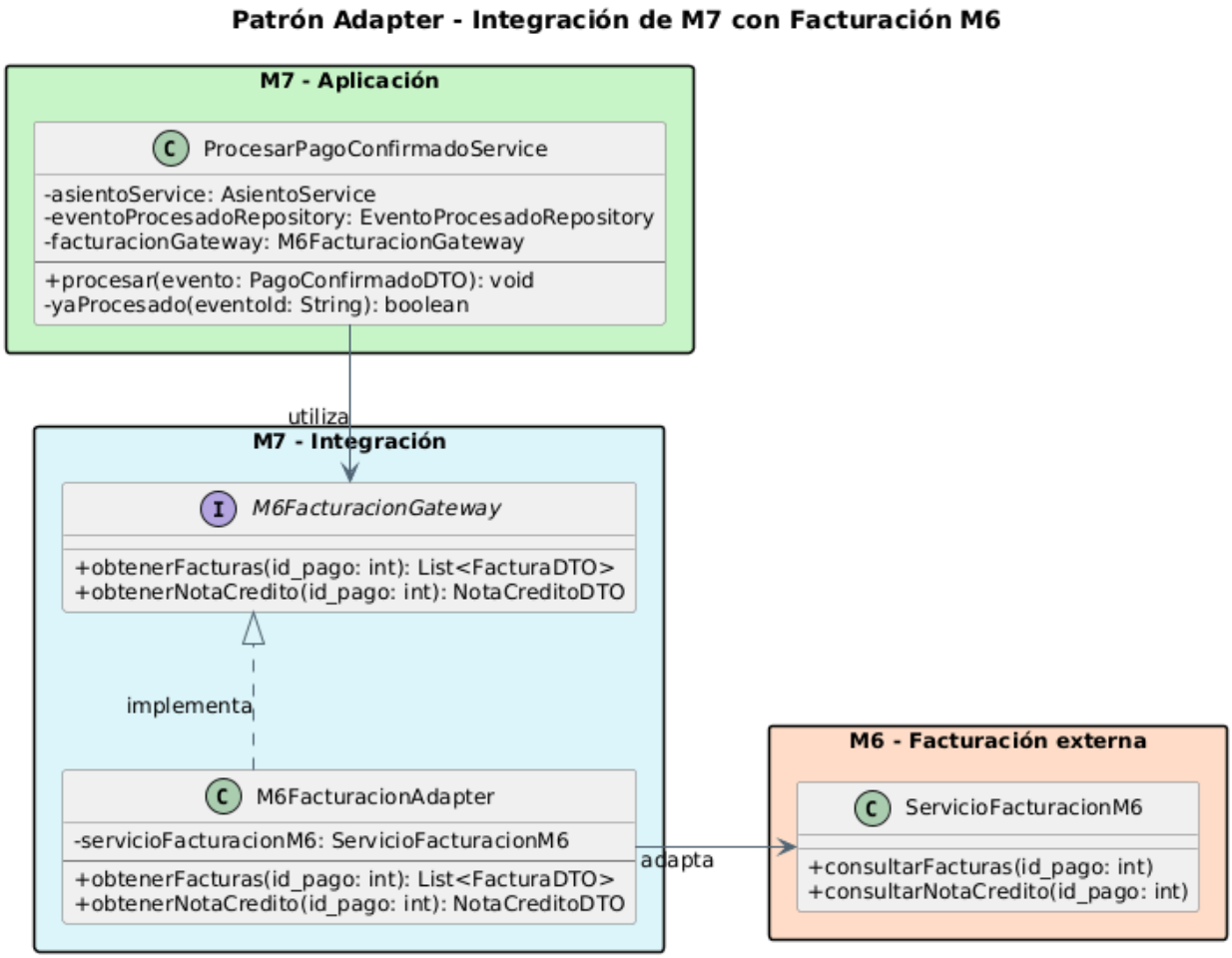
[Insertar fragmento del diagrama de clases donde se ve el patrón.]



7.3 Adapter

Categoría	Estructural
Problema del módulo que resuelve	El módulo de reportes utiliza un componente externo para generar reportes contables, pero la interfaz no coincide con la interfaz utilizada por el sistema.
Dónde se aplica	ProcesarPagoConfirmadoService FacturaciónGateway
Justificación	El patrón Adapter se utiliza para facilitar la integración entre ProcesarPagoConfirmadoService y FacturaciónGateway(M6), permitiendo que el módulo M7 utilice los servicios de facturación mediante una interfaz compatible con su diseño. De esta forma, se desacopla ProcesarPagoConfirmadoService de la implementación específica de M6, facilitando el mantenimiento y permitiendo realizar cambios en la integración sin afectar la lógica principal del módulo.
Alternativa descartada	Implementar directamente la comunicación con M6 dentro de ProcesarPagoConfirmadoService, sin utilizar un Adapter. Esta alternativa se descartó porque aumentaría el acoplamiento entre el servicio del M7 y la implementación de FacturaciónGateway, dificultando el mantenimiento y los posibles cambios en la integración con el módulo de Facturación.

[Insertar fragmento del diagrama de clases donde se ve el patrón.]



8. Prototipo de interfaz del módulo, validado

Cómo hacerlo

- Prototipar las pantallas de los casos de uso del módulo, empezando por los realizados en la sección 3.
- Usar React + Tailwind CSS dentro de la carpeta del módulo en el frontend, o una herramienta de prototipado cuyo resultado se pueda llevar a React sin cambios de diseño.
- Cumplir el requerimiento de adaptabilidad: operable desde 360 píxeles de ancho sin desplazamiento horizontal.
- Validar el prototipo con usuarios representativos de los actores del módulo y registrar qué se encontró y qué se cambió.

8.1 Pantallas del prototipo

Pantalla	Caso de uso	Actor	Ubicación (repositorio o enlace)
Tablero ejecutivo	Consulta de indicadores y resumen financiero	Comercio / usuario autorizado	frontend/src/modules/M7_Contabilidad/views/TableroEjecutivoView.jsx
Plan de cuentas	Administrar y consultar cuentas contables	Comercio / usuario autorizado	frontend/src/modules/M7_Contabilidad/views/PlanCuentasView.jsx

Libro diario	CU-002: generar asiento automático de venta; CU-003: registrar asiento automático de costo de ventas	Comercio / usuario autorizado; M5 inicia el registro por evento	frontend/src/modules/M7_Contabilidad/views/LibroDiarioView.jsx
Libro mayor	Consultar movimientos y saldo de una cuenta	Comercio / usuario autorizado	frontend/src/modules/M7_Contabilidad/views/LibroMayorView.jsx
Balance general	Consultar activos, pasivos y patrimonio	Comercio / usuario autorizado	frontend/src/modules/M7_Contabilidad/views/BalanceGeneralView.jsx
Estado de resultados	Consultar ingresos, costos, gastos y utilidad	Comercio / usuario autorizado	frontend/src/modules/M7_Contabilidad/views/EstadoResultadosView.jsx
Cuentas por cobrar y pagar	Consultar saldos pendientes	Comercio / usuario autorizado	frontend/src/modules/M7_Contabilidad/views/CuentasPorCobrarPagarView.jsx
Centros de costo	Consultar centros de costo y ejecución presupuestaria	Comercio / usuario autorizado	frontend/src/modules/M7_Contabilidad/views/CentrosCostoView.jsx

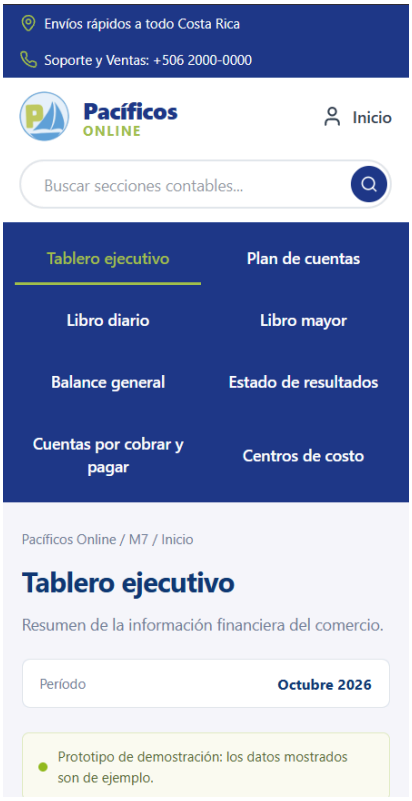
8.2 Validación

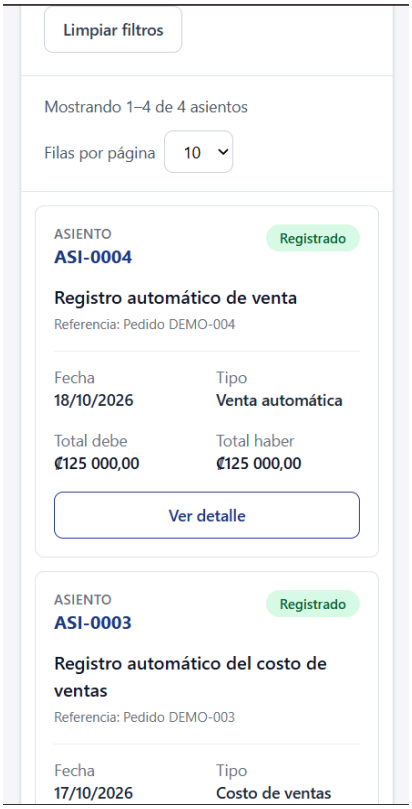
Fecha	21/10/2026
Participantes	Martin Chavez Angulo, coordinador del subgrupo SG7, realizó la validación del prototipo y comunicó las correcciones requeridas. Kristel Samanta De Franco Navarro aplicó los ajustes y verificó posteriormente la compilación y las pruebas automatizadas.
Método	Revisión del prototipo por parte de Martin Chavez Angulo (coordinador SG7), con observaciones sobre las dependencias de los dobles de prueba, la usabilidad móvil a 360 px y el acoplamiento del encabezado con M1. Luego se aplicaron los ajustes y se verificaron visualmente las pantallas; también se ejecutaron las pruebas automatizadas y la compilación.
Tareas evaluadas	Recorrer las ocho secciones; comprobar que el encabezado, buscador y navegación se adapten al ancho móvil; abrir y revisar el detalle de un asiento en Libro diario; revisar las tarjetas de Centros de costo; verificar que las tareas principales no requieran desplazamiento horizontal.

Hallazgo	Pantalla	Cambio realizado	Estado
Las tablas anchas no resultaban cómodas en 360 px y podían exigir desplazamiento horizontal.	Balance general, Estado de resultados, Libro diario, Libro mayor, Plan de cuentas, Cuentas por	Se adaptaron las vistas móviles mediante tarjetas, distribución flexible y estilos responsive; se comprobó visualmente el recorrido en móvil.	Aplicado; validación manual visual

Hallazgo	Pantalla	Cambio realizado	Estado
	cobrar y pagar, Centros de costo y Tablero ejecutivo.		
El encabezado reutilizaba recursos de M1, generando acoplamiento entre módulos.	Encabezado M7	Se creó IconoM7.jsx y se incorporó el logo en frontend/src/modules/M7_Contabilidad/assets/logo-pacificos.png .	Aplicado; compilación correcta
El detalle de asiento debía poder consultarse en una pantalla estrecha.	Libro diario — modal de detalle	Se verificó el detalle en formato vertical con desplazamiento dentro del modal y acción para cerrar.	Aplicado; revisado visualmente

[Insertar capturas del prototipo antes y después de la validación.]





Se incluye una captura del prototipo de M7 después de aplicar las correcciones solicitadas por el coordinador del subgrupo. No se conservaron capturas de la versión anterior.

9. Dobles de prueba de las dependencias externas

Permiten avanzar sin esperar a los otros subgrupos: sustituyen a cada módulo o servicio externo del que depende el módulo.

Cómo hacerlo

- Crear un doble por cada dependencia de la tabla «Lo que el módulo necesita» (1.1), incluidos los servicios externos simulados (pasarela, facturación) si aplican.
- El doble debe respetar la versión del contrato que imita: mismas operaciones, mismos campos y mismos errores.
- Tipos habituales: stub (respuestas fijas), fake (implementación simplificada que funciona), mock (verifica que se hicieron ciertas llamadas).
- Incluir casos de error en los dobles (por ejemplo, existencia insuficiente o pago rechazado), no solo el caso exitoso.
- Definir cómo se cambia del doble a la implementación real sin modificar el código del módulo (por ejemplo, por configuración).

Dependencia	Contrato y versión que imita	Tipo de doble	Escenarios que cubre	Ubicación en el repositorio	Se reemplaza cuando
M2 — Datos fiscales del comercio	ComerciosM7GatewayInterface. Contrato OpenAPI de M2 referenciado en contracts/m2-comercios/v1/openapi_M2_v1_2_1.yaml.	Stub	Datos fiscales disponibles; datos fiscales no disponibles (503); comercio no configurado (404). Estos escenarios forman parte de las 6 pruebas del gateway M2.	tests/Doubles/M7/ComerciosM7Stub.php; tests/Feature/M7_Contabilidad/ComerciosM7GatewayDoubleTest.php	Cuando se configure el gateway real en el contenedor de Laravel y se valide su correspondencia con el contrato de M2.
M2 — Comisión calculada del subpedido	ComerciosM7GatewayInterface. Operación para obtener la comisión calculada del subpedido.	Stub	Comisión disponible; comisión no disponible (503); subpedido sin comisión configurada (404). Estos escenarios forman parte de las 6 pruebas del gateway M2.	tests/Doubles/M7/ComerciosM7Stub.php; tests/Feature/M7_Contabilidad/ComerciosM7GatewayDoubleTest.php	Cuando se configure el gateway real y se valide su correspondencia con el contrato de M2.

M4 — Costos de productos vendidos	CostosProductosVendidosGatewayInterface. Contrato referenciado en contracts/m4-inventario/v1/openapi_M4_v1.1.0.yaml.	Stub	Costos disponibles; costos no disponibles (409); doble sin configurar (500); subpedido sin costos configurados (404).	tests/Doubles/M7/CostosProductosVendidosStub.php; tests/Feature/M7_Contabilidad/CostosProductosVendidosGatewayDoubleTest.php	Cuando se configure el gateway real y se valide su correspondencia con el contrato de M4.
M7 — Servicio contable	ContabilidadServiceInterface. Operación registrarAsientoDespacho(int pedidoId, float totalVenta, float costoVentas): bool.	Stub de PHPUnit	Una prueba comprueba que el contenedor de Laravel permite sustituir la interfaz por un stub y simular un retorno true. No comprueba la generación real de asientos.	tests/Feature/M7_Contabilidad/ContabilidadServiceDoubleTest.php	Cuando se implementen y prueben la lógica contable real, la persistencia de asientos y la integración correspondiente.

Nota: Las pruebas automatizadas de M7 finalizaron con 11 pruebas exitosas y 28 aserciones. Estas pruebas cubren los escenarios definidos para los dobles de los gateways de M2 y M4, además de la sustitución del servicio contable por un stub. Los resultados no demuestran la integración completa con los módulos reales ni la generación efectiva de asientos contables.